



Part—II

SystemTap Tutorial

Last month, we explored SystemTap and how it works. We also discussed Tapset libraries, and looked at a few examples. Let's proceed from there.

I am sure you have gone through the examples given on the SystemTap website (<http://sourceware.org/systemtap/examples/index.html>) and looked at other resources I mentioned in the *References* section of the last article. I will not try to cover all the topics mentioned there.

To collect statistical data, SystemTap supports aggregate variables. These are important when data needs to be collected quickly and in large volumes. The “<<<” operator is used for aggregation. The left operand specifies a scalar or array-index l-value, which must be declared global. The right operand is a numeric expression.

Let's update the example (*stapcount.stp*) we saw last time, to use aggregation:

```
global syscalls
probe syscall.* { syscalls[name] <<< 1 }
probe timer.s(10) {
    foreach(name in syscalls- limit 5)
        printf("%8d %s\n", @count(syscalls[name]), name)
    delete syscalls
}
```

You might wonder that if it serves the same purpose, why should we use aggregation?

The collected data can later be extracted via different extractor functions like `@count(var)`,

`@min(var)`, `@max(var)`, etc. You can also generate nice-looking histograms with the `@hist_log` and `@hist_linear` functions. For more information, please refer to `/usr/share/doc/systemtap-<version>/langref.pdf`.

Call tracing

Wouldn't it be good to know the flow of execution through the code, while debugging a problem?

SystemTap makes this very simple. Let's say you want to know all the “ext3”-related functions that are called when you create a file on an ext3 file-system.

As my installed distribution uses the (default) ext4 file-system on the root partition, I created a loop-back device, and formatted it as ext3. The call trace can provide a lot of information, which might be difficult to follow. But by having just one device formatted with ext3, I have narrowed down my focus. So, here is the script:

```
probe kernel.function("**@fs/ext3/**") {
    printf ("%s -> %s\n", thread_indent(1),
    probefunc())
}

probe kernel.function("**@fs/ext3/**").return {
    printf ("%s <- %s\n", thread_indent(-1), probefunc())
}
```